

ACKNOWLEDGEMENT

I would like to take this opportunity to express my sincere gratitude and heartfelt appreciation to my project guide for their invaluable guidance, continuous encouragement, constructive feedback, and support throughout the development of Verse Hana. Their guidance and suggestions played an important role in helping me plan, develop, and successfully complete this project.

I would also like to extend my sincere thanks to Purbanchal University School of Engineering for providing me with the academic environment, facilities, resources, and opportunities necessary for the successful completion of this project. I would also like to take this opportunity to express our sincere gratitude and heartfelt thanks to our project guide, Er. Santosh Sah, Coordinator, for his invaluable guidance throughout this work. The knowledge and skills gained throughout my academic journey have been instrumental in designing and developing Verse Hana.

Finally, I would like to express my sincere appreciation to my friends, colleagues, and everyone who supported and encouraged me directly or indirectly throughout the planning, development, testing, and completion of this project. Their support and encouragement have been valuable in making this project possible.

Abstract

The project titled “**Verse Hana – Emotion-Based Music Discovery Platform**” focuses on developing a modern, responsive, and user-friendly web application for discovering and listening to music based on moods and emotions. Conventional music platforms primarily depend on song titles, artists, genres, and keyword-based searches. Although these methods are useful, users may sometimes know how they feel rather than exactly what song or genre they want to listen to. Verse Hana was developed to provide an alternative discovery experience by organizing music around predefined moods such as Happy, Calm, Rain, Night, Love, Focus, Party, and Sad.

The platform provides essential features including user registration and login, mood-based music discovery, music search, music playback, lyrics, likes, playlists, listening history, user profiles, and an administrative dashboard. The system also integrates the **Audius API** to retrieve and stream music while using **MongoDB Atlas** for persistent application data. Password-reset functionality is supported through **Brevo** email services.

The frontend of the system is developed using **React.js, Vite, Tailwind CSS, Framer Motion, and React Icons**, while the backend is implemented using **Node.js and Express.js**. MongoDB with Mongoose is used for database management.

Throughout the development process, emphasis was placed on creating a cinematic and responsive user interface, maintaining smooth navigation, integrating external services, and providing a practical music discovery experience. The project demonstrates the application of full-stack web development concepts including frontend development, REST APIs, database management, authentication, authorization, third-party API integration, responsive UI design, and deployment.

Verse Hana currently uses predefined mood categories rather than artificial intelligence or machine-learning-based emotion detection. However, the system provides a foundation for future enhancements such as intelligent recommendation systems, emotion detection, contextual recommendations, and personalized music discovery.

Table of contents

S.N.	Contents	Page
1	Acknowledgement	1
2	Abstract	2
3	Chapter 1: Introduction	4
4	Chapter 2: Literature Review	9
5	Chapter 3: Analysis of Issues and Solution	10
6	Chapter 4: Design Specification and Implementation	12
7	Chapter 5: Testing and Implementation	18
8	Chapter 6: Conclusion and Future Work	20
9	References	22
10	Bibliography	22
11	Appendix A: System Screenshots	23
12	Appendix B: System Diagrams	27
13	Appendix C: Selected Code Snippets	27
14	Appendix D: Testing Results and User Feedback	30
15	Closing Summary	32

Chapter 1: Introduction

1.1 Background and Significance of the Project

Music streaming platforms have made a large amount of music easily accessible to users. However, conventional music discovery mainly depends on song titles, artists, genres, keywords, or existing recommendations. Users may sometimes want music according to their current mood or situation rather than a specific song or artist.

Verse Hana is an emotion-based music discovery platform developed to provide a simpler way of discovering music according to predefined moods. The platform provides mood categories such as **Happy, Calm, Rain, Night, Love, Focus, Party, and Sad**, allowing users to explore music based on how they feel.

Verse Hana also combines common music platform features such as search, music playback, lyrics, liked songs, playlists, listening history, user accounts, and an administrator dashboard. The system is developed using React and Vite for the frontend, Node.js and Express.js for the backend, MongoDB for data storage, Audius API for music services, and Brevo for password-reset email functionality.

Problem Statement

Users often have difficulty finding suitable music when they want songs for a particular mood. Searching individually for songs and creating suitable playlists can also be time-consuming. Existing platforms provide large music libraries and recommendation features, but users may still need to rely on search terms or automatically generated recommendations.

Therefore, there is a need for a simple platform that allows users to discover music through clearly defined mood categories while also providing basic music management features in one system.

Proposed Solution

Verse Hana provides a web-based platform where users can discover music through predefined mood categories. Users can search for songs or artists, play music, view lyrics, like songs, create playlists, and access their listening history.

The system also provides authentication and authorization for users and administrators. MongoDB is used to store application data, while Audius provides music-related data and streaming services. Password recovery is supported through OTP-based email verification using Brevo.

1.2 Objectives and Scope

The main objectives of this project are:

- To design and develop a responsive and user-friendly music discovery platform.
- To allow users to discover music according to predefined moods.
- To allow users to register, log in, search for songs, and listen to music.
- To provide a centralized music player for smooth music playback.
- To allow users to like songs and create playlists.
- To maintain listening history for users.
- To provide lyrics and song-related information where available.
- To provide a secure backend system for storing and managing user and music-related data.
- To implement authentication and role-based authorization.
- To create an admin dashboard for managing platform data and users.
- To integrate an external music service using the Audius API.
- To demonstrate the use of full-stack technologies including React, Node.js, Express.js, MongoDB, and related technologies.

Scope:

The scope of this project includes:

- Development of a complete web-based music discovery platform.

- Frontend development using React, Vite, Tailwind CSS, Framer Motion, and React Icons.
- Backend implementation using Node.js and Express.js.
- Integration of MongoDB for storing user, playlist, like, history, and other application data.
- Integration of the Audius API for music-related services.
- User authentication and role-based functionalities.
- Mood-based music discovery.
- Music search and playback.
- Playlist and liked-song management.
- Listening history.
- Password-reset functionality through email OTP.
- Administrative dashboard.
- Responsive and modern user interface.

The system focuses on core music discovery and management features while keeping the architecture suitable for future upgrades such as intelligent recommendations, machine-learning-based emotion detection, contextual recommendations, and mobile applications.

1.3 Project Features

The major features of Verse Hana are:

- **User Authentication:** Registration, login, logout and password reset.
- **Mood-Based Discovery:** Music browsing through predefined moods.
- **Music Search:** Search songs and artists.
- **Music Player:** Play, pause, seek and control volume.
- **Liked Songs:** Save preferred songs.
- **Playlists:** Create and manage personal playlists.
- **Listening History:** View recently played music.
- **Lyrics:** Display available song lyrics.
- **User Profile:** Manage user information.
- **Admin Dashboard:** Manage and monitor system information.
- **External API Integration:** Integration with Audius and Brevo.

- **Responsive UI:** Support for different screen sizes.
- **Theme Support:** Light and dark interface.

1.4 Summary and Project Organization

This report outlines the complete development process of **Verse Hana – Emotion-Based Music Discovery Platform**, beginning from the identification of problems in conventional music discovery to the design, implementation, testing, and evaluation of a functional web application. The introduction highlights the motivation, objectives, scope, and major features of the system, establishing the foundation for the subsequent chapters.

The entire project is organized into structured chapters to ensure clarity and smooth flow of information:

- **Chapter 1 – Introduction:**

This chapter provides a comprehensive introduction to the project. It explains the background and significance of developing an emotion-based music discovery platform and highlights the problems users face while searching for suitable music. It includes the project objectives, scope, major features, and the rationale behind choosing the technologies used. This chapter establishes the foundation of the report and outlines what the system aims to accomplish.

- **Chapter 2 – Literature Review:**

This chapter presents an overview of existing music streaming and music discovery systems and the technological approaches relevant to the project. It compares existing platforms and their approaches to music search, recommendation, categorization, and streaming to identify their strengths and limitations. The review highlights the gaps that justify the development of Verse Hana.

- **Chapter 3 – Analysis of Issues and Solution:**

This chapter analyzes the major problems associated with conventional music discovery systems and explains how Verse Hana provides solutions to these issues.

It discusses mood-based discovery, centralized music access, user interaction, playlist management, authentication, and other system functionalities.

- **Chapter 4 – Design Specification and Implementation:**

This chapter describes the complete technical development of the system. It includes system architecture, data flow design, use case diagrams, database design, and user interface planning. The frontend development using React and related technologies is explained, while the backend section covers Node.js, Express.js routing, API structure, middleware, authentication mechanisms, and database operations using MongoDB.

- **Chapter 5 – Testing and Implementation:**

This chapter explains how Verse Hana was implemented and tested to ensure that it functions correctly and reliably. It includes frontend, backend, database, authentication, API, responsive UI, and functional testing. Test cases and their results are also presented.

- **Chapter 6 – Conclusion and Future Work:**

This chapter summarizes the completed project and discusses its achievements and limitations. It also presents possible future enhancements such as intelligent recommendation systems, machine-learning-based emotion detection, contextual music discovery, improved personalization, mobile applications, and analytics.

- **References:**

This section lists the documentation, technical resources, APIs, research materials, and online resources consulted during the development of the system.

- **Bibliography:**

Provides additional learning resources and supporting materials relevant to the technologies and concepts used in the project.

- **Appendices:**

Includes supplementary materials such as system screenshots, diagrams, sample data, source-code snippets, API examples, and testing documentation.

This systematic organization helps in understanding the Verse Hana project holistically while clearly presenting each stage of its development.

Chapter 2: Literature Review

2.1 Previous System or Work Study

Music discovery has traditionally been based on radio, television, personal recommendations, song titles, artists, and genres. With the development of online music platforms, users can now access large music libraries through web and mobile applications.

However, conventional discovery methods mainly depend on searching for specific songs, artists, genres, or recommendations. Users looking for music according to a particular mood may need to manually search and create playlists. Verse Hana addresses this issue by providing predefined mood-based categories for simpler music discovery.

2.2 Review of Existing Music Streaming and Discovery Systems

a. Spotify

Provides music streaming, search, playlists, and personalized recommendations. Its large collection provides many choices, but users may still depend on search and recommendation features to find music matching a specific mood.

b. YouTube Music

Provides music search, streaming, playlists, and recommendations. Its large content library makes music accessible but can also make specific mood-based discovery less direct.

c. Apple Music

Provides music streaming, curated playlists, search, and recommendations. It provides various ways to discover music but primarily focuses on conventional music browsing and recommendation.

d. Audius

Provides music-related services and APIs that can be integrated into applications. Verse Hana uses Audius as an external music service while providing its own mood-oriented interface and application features.

Chapter 3: Analysis of Issues and Solution

3.1 Introduction

This chapter analyzes the main problems associated with conventional music discovery and presents the proposed solution. The analysis focuses on making music discovery simpler while providing essential music management features in a single platform.

3.2 Analysis of Existing Issues

The major issues identified are:

- **Mood-based discovery:** Users may want music according to their mood rather than a specific artist or song.
- **Time-consuming search:** Finding suitable songs manually can take time.
- **Manual playlist creation:** Users may need to search and organize songs individually.
- **Fragmented features:** Search, playlists, likes, history, and lyrics may not always be presented together in a simple interface.
- **Large music libraries:** A large number of available songs can make finding suitable music difficult.
- **User management:** A music platform requires secure authentication and proper management of user data.

3.3 Proposed Solution

Verse Hana is proposed as a centralized web-based music discovery and management platform that addresses the limitations identified in conventional music discovery systems. The main purpose of the proposed system is to provide users with a simple way to discover music according to their current mood instead of depending only on song names, artists, or genres.

The system provides a set of predefined mood categories such as **Happy, Calm, Rain, Night, Love, Focus, Party, and Sad**. Users can select a mood and explore the available songs associated with that category. This approach reduces the effort required to manually search for suitable songs and provides a more direct method of music discovery.

In addition to mood-based discovery, Verse Hana provides common music-related functionalities within a single platform. Users can search for songs and artists, play available music, view lyrics, like songs, create and manage playlists, and view their listening history. These features allow users to discover and organize their preferred music without having to use separate systems for different activities.

3.4 Expected Benefits

The proposed system provides the following benefits:

- Easier mood-based music discovery.
- Reduced effort in searching for suitable songs.
- Centralized music management.
- Personalized playlists and liked songs.
- Access to listening history.
- Secure user authentication.
- Responsive and user-friendly interface.
- Integration with external music services.
- Efficient music search and playback.
- Controlled access to administrative features.
- Support for future intelligent music recommendations.

Chapter 4: Design Specification and Implementation

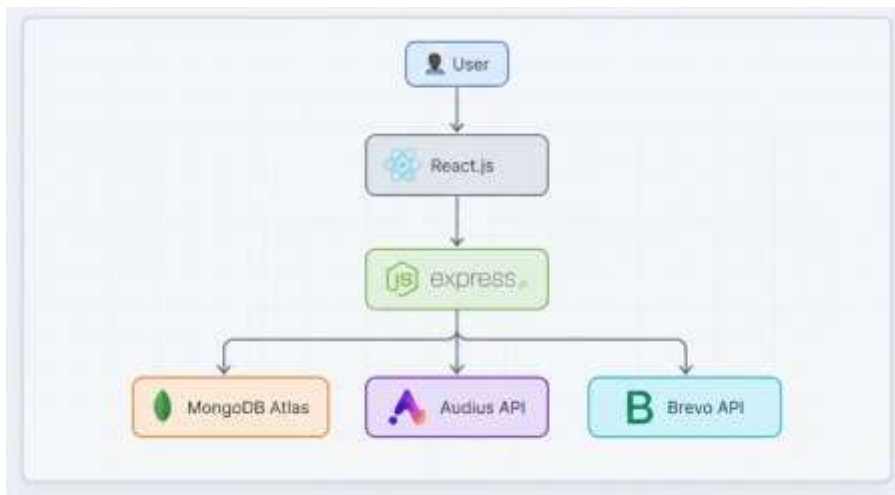
4.1 System Architecture

Verse Hana follows a client-server architecture consisting of the frontend, backend, database, and external services.

The major components of the system are:

- **Frontend:** React with Vite, Tailwind CSS, Framer Motion, and React Icons.
- **Backend:** Node.js and Express.js for API handling and application logic.
- **Database:** MongoDB Atlas for storing application data.
- **Music Service:** Audius API for music-related data and streaming.
- **Email Service:** Brevo API for password-reset emails.

The general architecture of Verse Hana is:



The frontend communicates with the Express backend through REST APIs. The backend processes user requests, manages application data, and communicates with external services when required.

4.2 Data Flow Diagram

The major data flows in Verse Hana include:

- User registration and login.
- Music search and discovery.
- Mood-based music selection.
- Music playback.
- Playlist creation and management.
- Liking and unlinking songs.
- Listening history management.
- Password reset through OTP.
- Administrator operations.

The user interacts with the frontend, which sends requests to the backend. The backend processes these requests and communicates with MongoDB or external APIs as required.

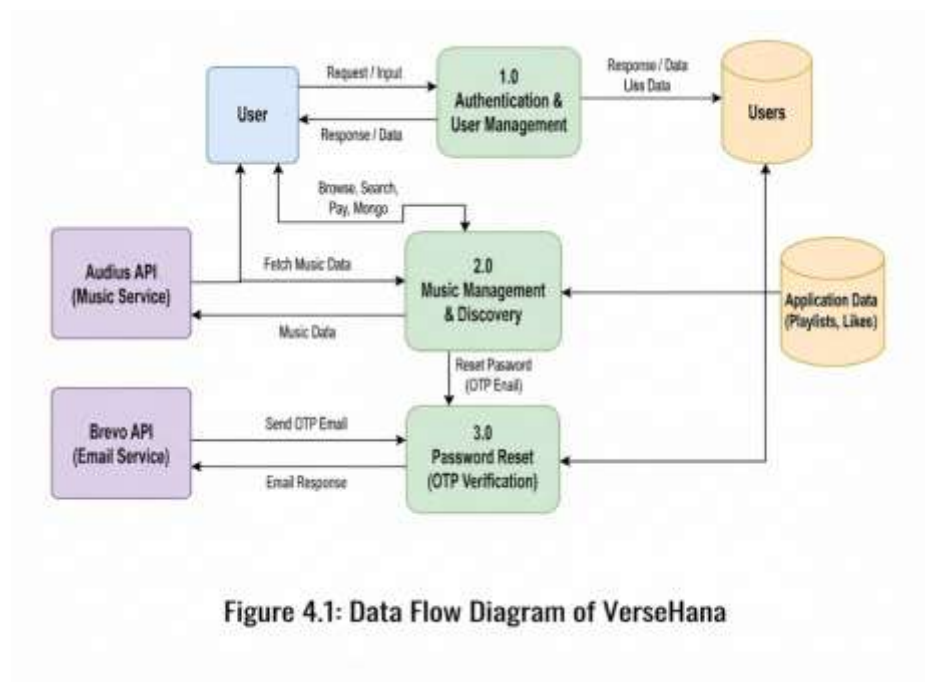


Figure 4.1: Data Flow Diagram of VerseHana

4.3 Use Case Diagram

Verse Hana has three main types of actors:

Visitor

- View home page.
- Browse moods.
- Search music.
- Register.
- Login.

Registered User

- Browse mood-based music.
- Search songs and artists.
- Play music.
- View lyrics.
- Like songs.
- Create and manage playlists.
- View listening history.
- Manage profile.
- Reset password.

Administrator

- Login to admin panel.
- View dashboard.
- Manage users.
- Manage relevant system information.

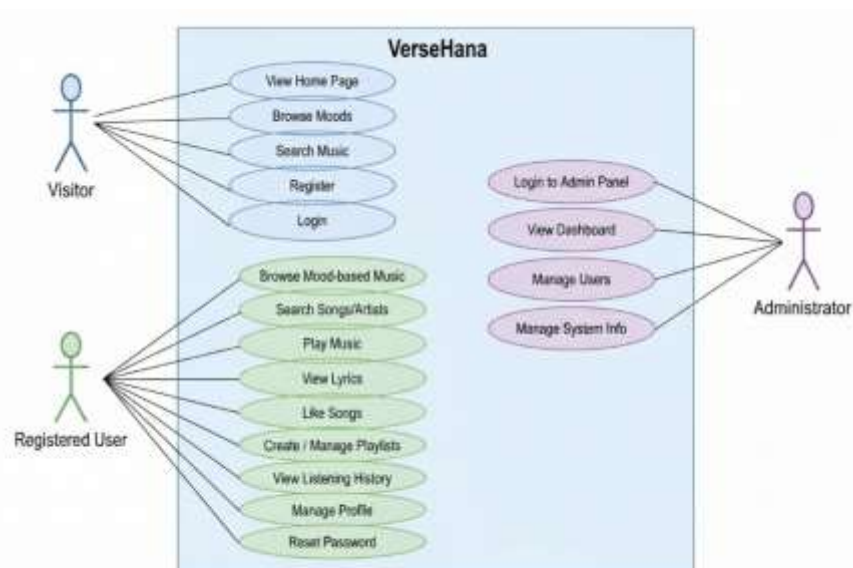


Figure 4.2: Use Case Diagram of VerseHana

4.4 Database Design

MongoDB Atlas is used to store the application's persistent data. Mongoose is used in the backend to define schemas and interact with MongoDB.

The major data maintained by the system includes:

- **Users:** Stores user account and authentication-related information.
- **Playlists:** Stores user-created playlists and their associated songs.
- **Likes:** Stores songs liked by users.
- **History:** Stores recently played music.
- **Music-related Data:** Stores required music information used by the application.

MongoDB was selected because its document-based structure provides flexibility for managing the application's data.

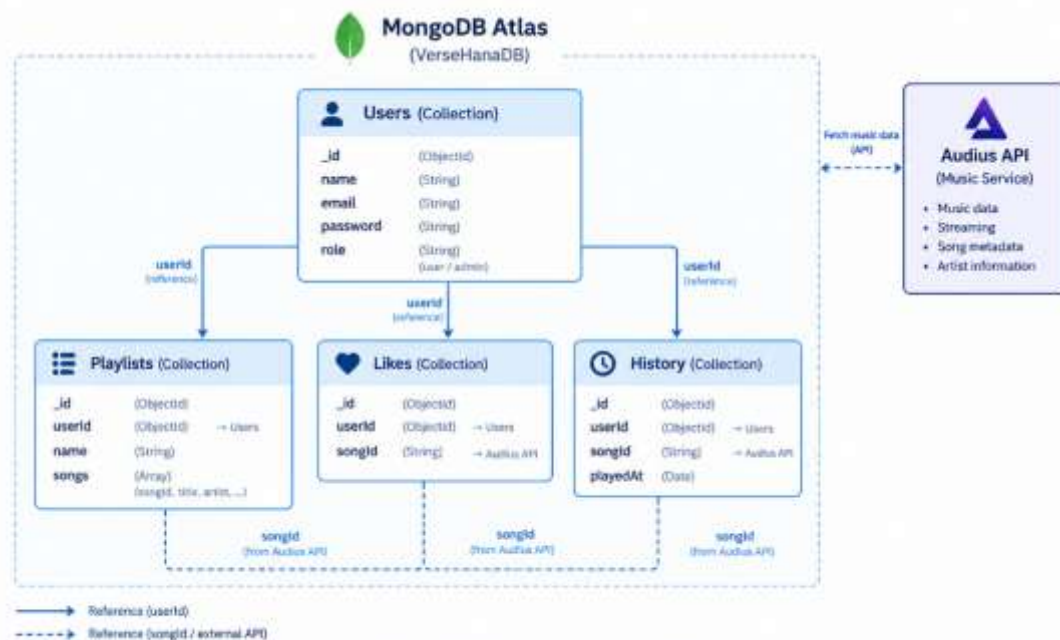


Figure 4.3: MongoDB Database Structure of VerseHana

4.5 Frontend Implementation

The frontend of Verse Hana is implemented using React and Vite. Tailwind CSS is used for styling, while Framer Motion is used for animations and interactive effects.

The main frontend components include:

- Home page.
- Discover page.
- Mood pages.
- Search interface.
- Music player.
- Lyrics interface.
- Playlist pages.
- Liked songs.
- Recently played.
- User profile.
- Authentication pages.
- Admin dashboard.

The interface also supports responsive layouts and light/dark themes.

4.6 Backend Implementation

The backend is developed using Node.js and Express.js. It provides REST APIs that connect the frontend with the database and external services.

The backend handles:

- User authentication.
- Authorization.
- Music-related requests.
- Playlist operations.
- Likes.
- Listening history.
- Admin operations.
- Password reset.
- External API communication.

The backend separates application logic from the frontend and provides controlled access to database operations.

4.7 Authentication and Security

Verse Hana uses authentication and authorization to protect user accounts and restricted features.

The main security mechanisms include:

- Password hashing using bcrypt/bcryptjs.
- JWT-based authentication.
- HTTP-only cookie for storing the authentication token.
- Role-based authorization for administrative functions.
- OTP-based password recovery.
- Environment variables for sensitive configuration.
- Backend-side authorization for protected operations.
- CORS configuration for frontend-backend communication.

The password-reset system generates a temporary OTP and sends it through email. The OTP is time-limited and verified before allowing the user to reset the password.

Chapter 5: Testing and Implementation

5.1 Introduction

Testing was performed to verify that the major features of Verse Hana work correctly and that the frontend, backend, database, and external services communicate properly.

The testing process focused mainly on authentication, mood discovery, music search, playback, playlists, likes, history, and administrative functions.

5.2 Implementation

The system was implemented as a full-stack web application using React for the frontend and Node.js with Express.js for the backend.

The frontend provides the user interface, while the backend manages application logic and API requests. MongoDB Atlas stores application data, and external services are used for music and email functionality.

The major implemented modules are:

- User authentication.
- Mood-based discovery.
- Music search.
- Music player.
- Lyrics.
- Likes.
- Playlists.
- Listening history.
- User profile.
- Password reset.
- Admin dashboard.

5.3 Testing

The system was tested using functional and integration testing. Major features were tested using different valid and invalid inputs to verify expected behavior.

Test Area	Test Case	Expected Result
Registration	Valid user details	Account created
Login	Valid credentials	User logged in
Login	Invalid credentials	Error displayed
Mood	Select a mood	Related songs displayed
Search	Search song/artist	Results displayed
Player	Play/pause song	Playback works
Likes	Like a song	Song added to liked list
Playlist	Create playlist	Playlist created
History	Play a song	Song added to history
Admin	Admin login	Dashboard accessible
Authorization	Normal user opens admin	Access restricted

5.4 Test Result

The major functional areas of the system were checked during development. The tests focused on ensuring that user actions produce the expected results and that restricted features are protected.

The testing confirmed the interaction between the major system components, including the React frontend, Express backend, MongoDB database, and external services.

Chapter 6: Conclusion and Future Work

6.1 Conclusion

Verse Hana is an emotion-based music discovery platform developed to make music discovery simpler and more convenient for users. Instead of depending only on traditional searches such as song titles, artists, or genres, the system provides predefined mood categories such as Happy, Calm, Rain, Night, Love, Focus, Party, and Sad. This allows users to explore music according to their current mood or preference.

The system provides major music-related features including music search, mood-based discovery, music playback, lyrics, liked songs, playlists, listening history, and user profile management. It also includes user authentication, authorization, password recovery using OTP, and an administrative dashboard. MongoDB Atlas is used for application data management, while the Audius API provides music-related data and streaming services. Brevo is used for sending password-reset OTP emails.

The project demonstrates the development of a complete full-stack web application using React, Vite, Tailwind CSS, Framer Motion, React Icons, Node.js, Express.js, MongoDB, and external APIs. The developed system provides a foundation for improving music discovery and can be extended with more advanced personalization features in the future.

6.2 Future Work

Although Verse Hana provides the major features required for mood-based music discovery, several improvements can be made in future versions. Possible future enhancements include:

- **AI-based Emotion Detection:** Detect emotions from text, voice, or other user inputs and suggest suitable music automatically.
- **Machine Learning Recommendations:** Develop a recommendation system based on user listening behavior, likes, playlists, and history.
- **Context-Aware Recommendations:** Consider activities and situations such as studying, exercising, relaxing, travelling, or sleeping.

- **Advanced Playlist Generation:** Automatically generate playlists based on mood, activity, and user preferences.
- **Improved Personalization:** Provide recommendations that become more accurate as the user interacts with the platform.
- **Mobile Application:** Develop dedicated Android and iOS applications for easier access.
- **Enhanced Admin Features:** Provide more detailed system statistics, user management, and content monitoring.
- **Expanded Music Services:** Support additional music sources and APIs to increase the available music library.

6.3 Summary

In conclusion, Verse Hana provides the foundation for a modern and user-oriented music discovery environment. The platform combines mood-based discovery with common music streaming features such as search, playback, playlists, liked songs, and listening history.

With future enhancements such as artificial intelligence, machine-learning-based recommendations, contextual discovery, and mobile accessibility, the platform has the potential to become a more intelligent and personalized music discovery system.

The project demonstrates how modern full-stack technologies can be combined to address a practical problem in digital music discovery while providing a scalable foundation for future development.

References

- Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7th ed.). O'Reilly Media.
- Banks, A., & Porcello, E. (2020). *Learning React: Modern Patterns for Developing React Apps* (2nd ed.). O'Reilly Media.
- Hahn, E. (2016). *Express in Action: Writing, Building, and Testing Node.js Applications*. Manning Publications.
- Chodorow, K. (2013). *MongoDB: The Definitive Guide* (2nd ed.). O'Reilly Media.

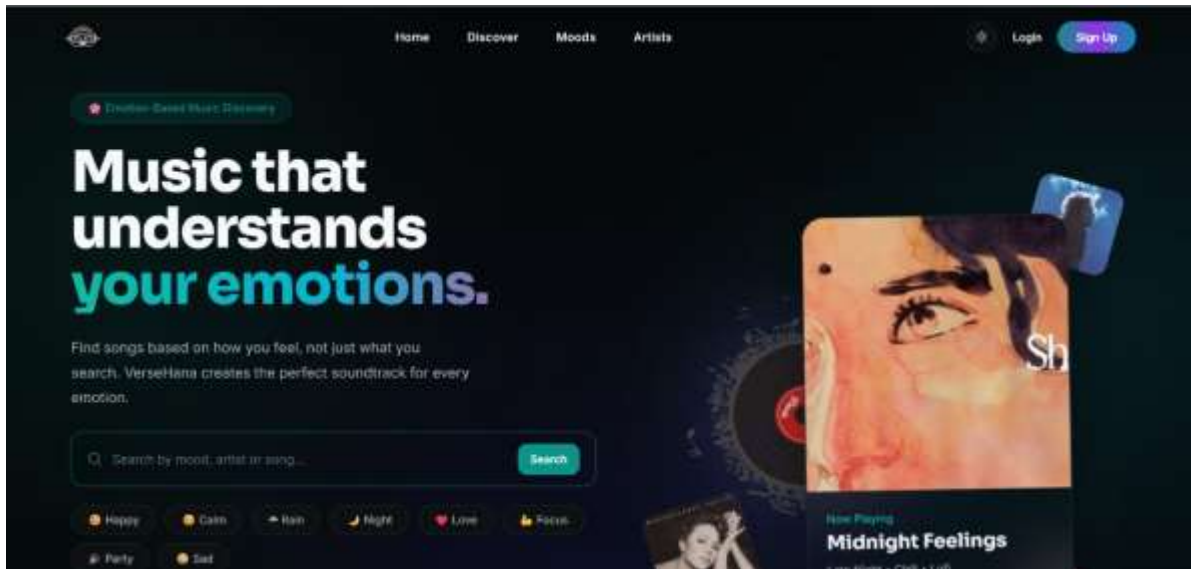
Bibliography

- React Documentation – React components, hooks, state management, and frontend development.
- Vite Documentation – Vite project setup, development server, and build configuration.
- Tailwind CSS Documentation – Utility-first CSS framework and responsive interface development.
- Framer Motion Documentation – Animation and interaction implementation in React applications.
- Node.js Documentation – Server-side JavaScript runtime and application development.
- Express.js Documentation – Routing, middleware, REST API, and backend development.
- MongoDB Documentation – Database design, collections, documents, and data modeling.
- Mongoose Documentation – MongoDB object modeling and schema management for Node.js.
- Audius Developer Documentation – Music data, artist information, and streaming API integration.
- Brevo API Documentation – Email service and password-reset OTP implementation.
- MDN Web Docs – JavaScript, HTTP, web APIs, and web development concepts.
- W3Schools – Supporting tutorials and references for HTML, CSS, JavaScript, and web development.

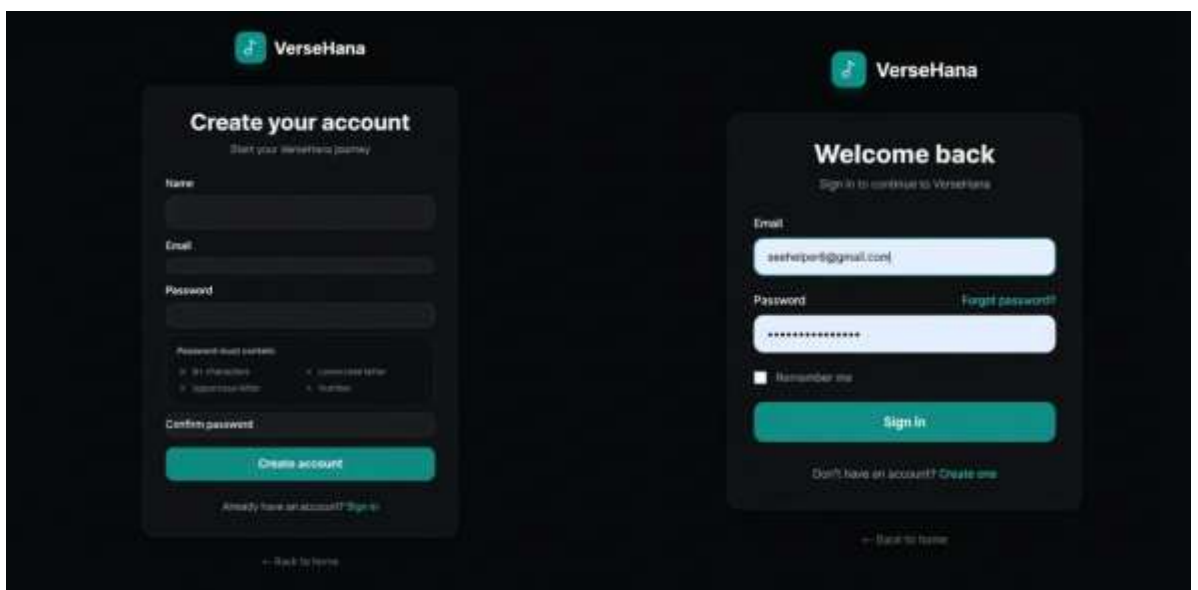
Appendices

Appendix A: System Screenshots

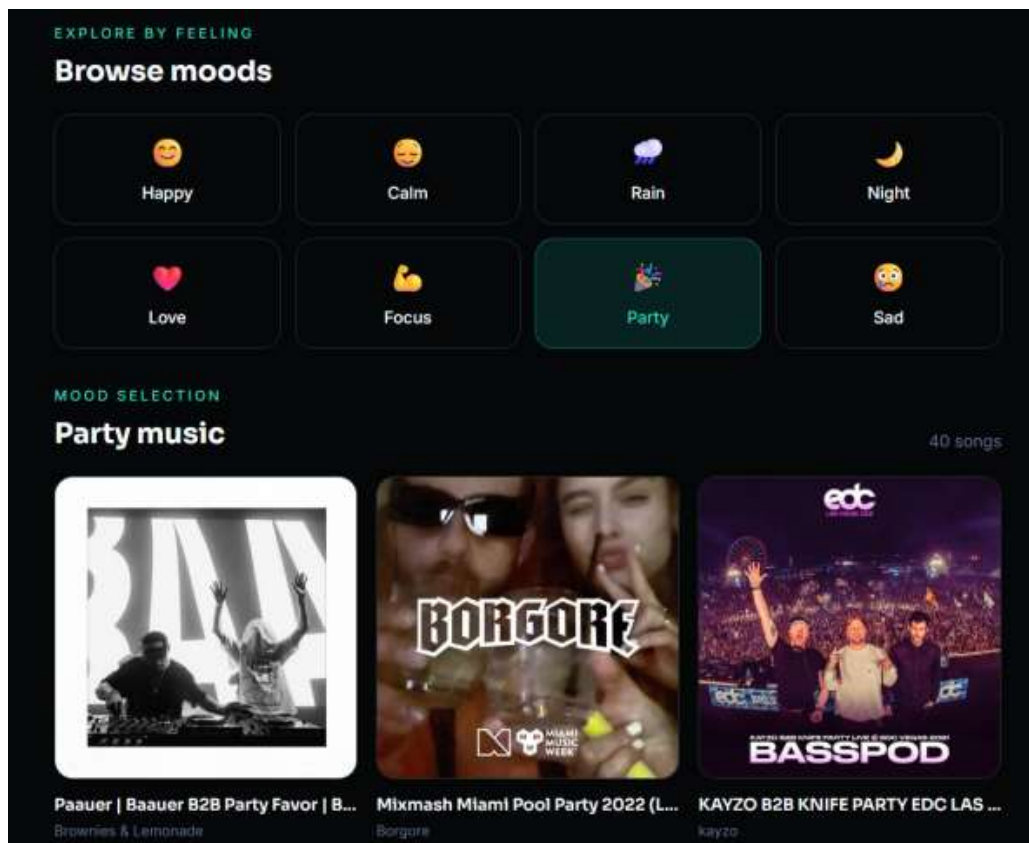
- Home Page



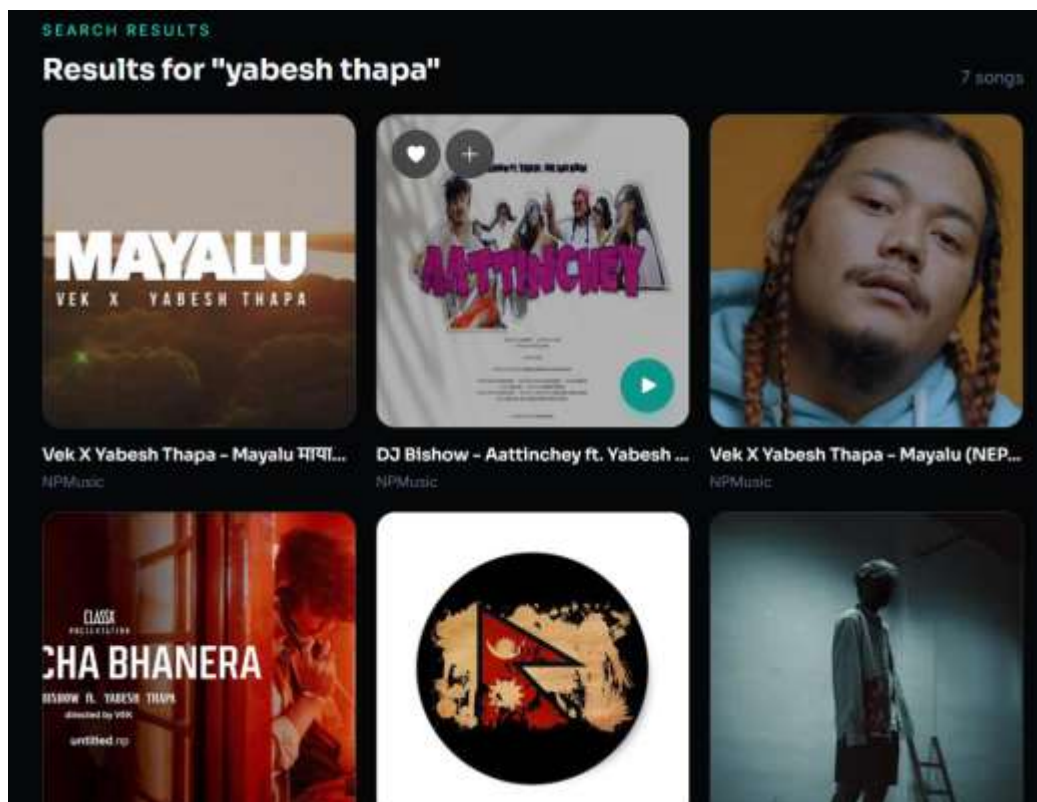
- Login/Register Page



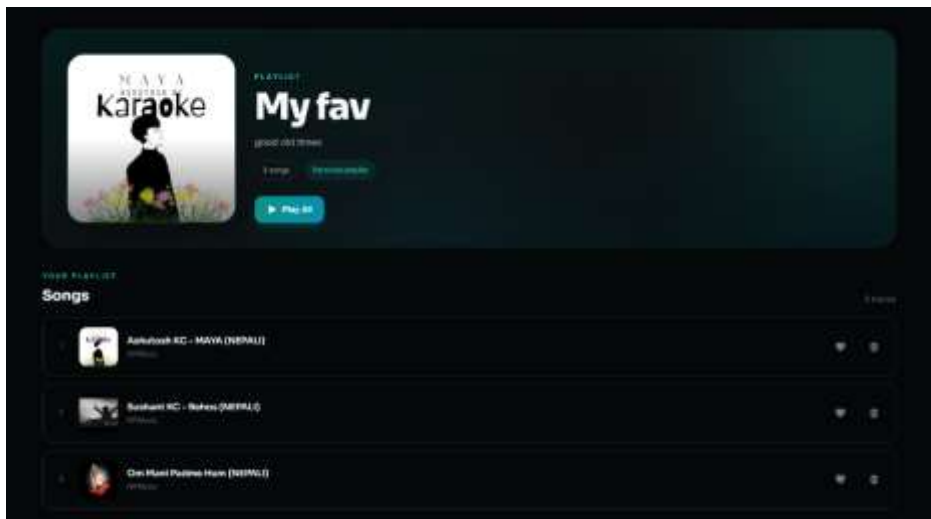
- Mood Discovery Page



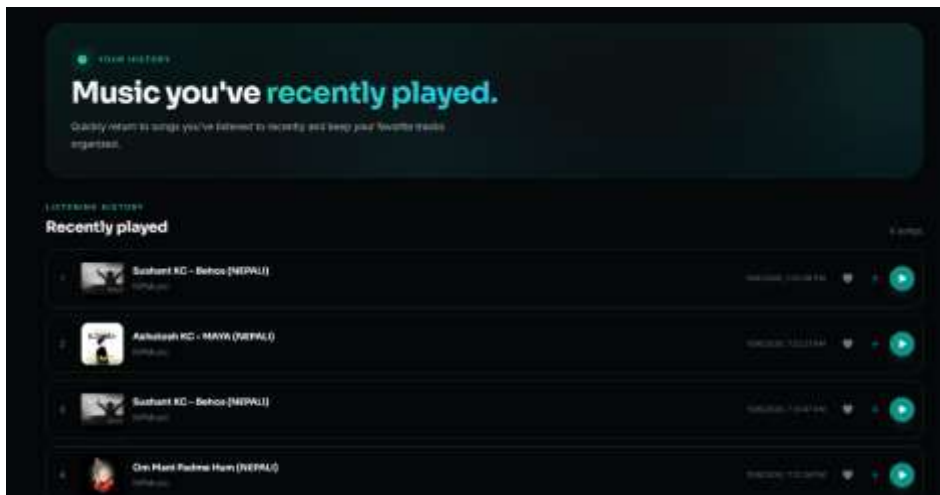
- Search Results



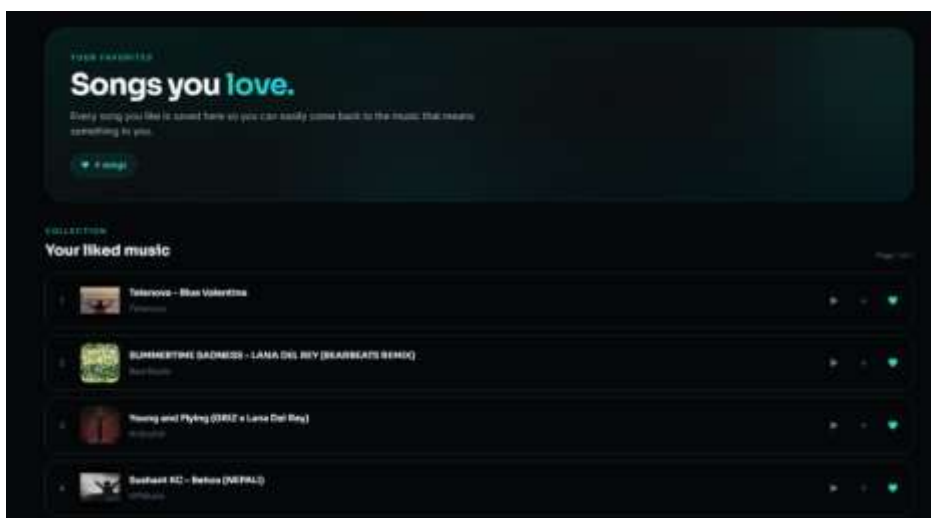
- **Playlist Page**



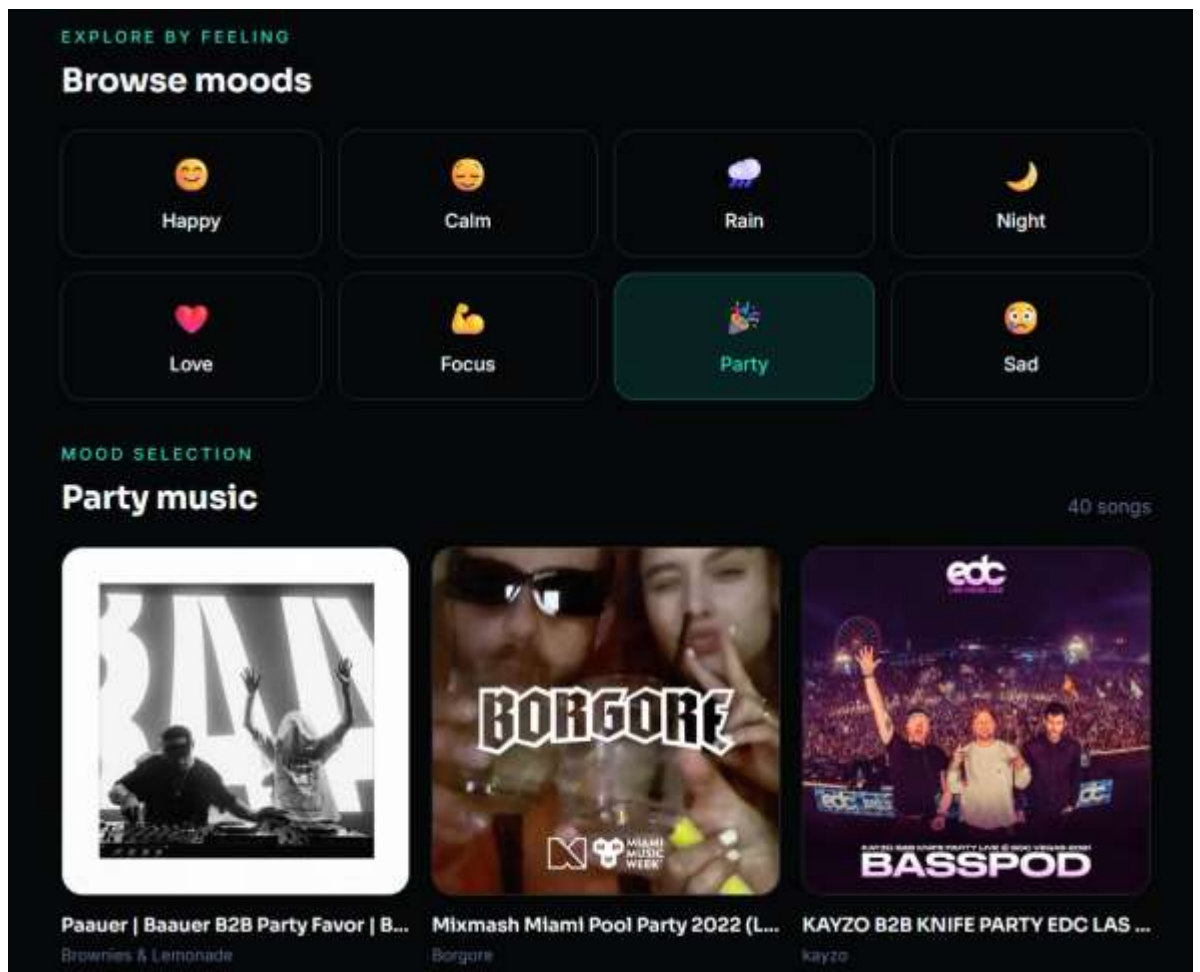
- **Recently Played**



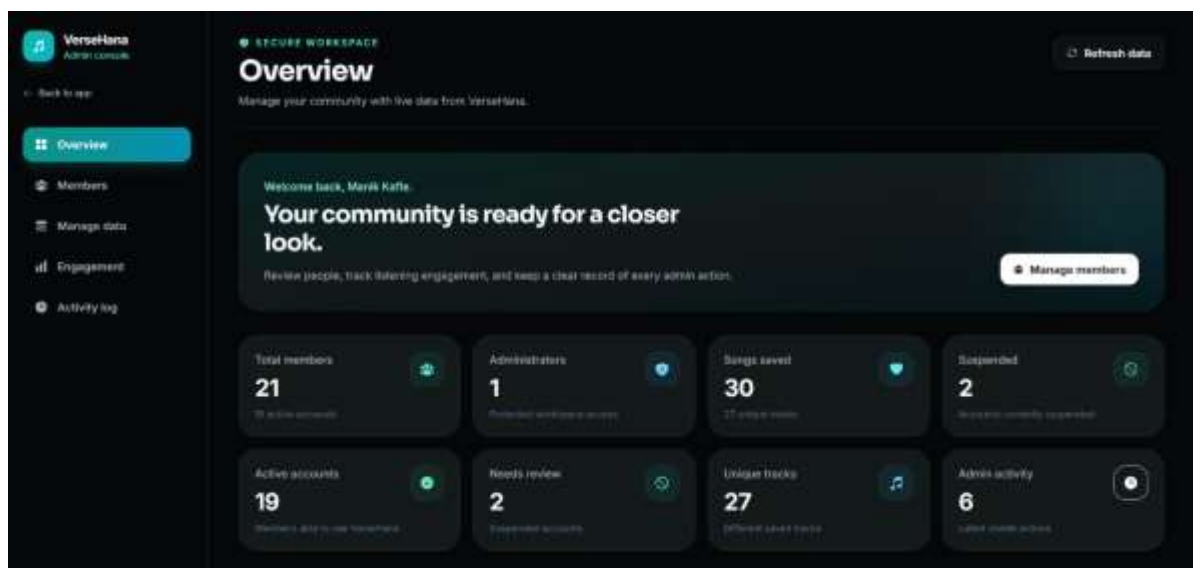
- **Liked Songs**



- User Profile



- Admin Dashboard



Appendix B: System Diagrams

- System Architecture Diagram
- Data Flow Diagram (DFD)
- Use Case Diagram
- MongoDB Database Structure

Appendix C: Selected Code Snippets

- User Authentication

```
const hashedPassword =
  await bcrypt.hash(
    password,
    12
  );

const user =
  await User.create({
    name,
    email,
    password:
      hashedPassword,
    role: "user",
    status: "active",
  });

return res.status(201).json({
  message:
    "Account created successfully.",
  user: {
    id: user._id,
    name: user.name,
    email: user.email,
    role: user.role,
    status: user.status,
  },
});
```

- JWT Authentication

```
const createToken = (user) =>
  jwt.sign(
    {
      id: user._id,
      role: user.role,
      authVersion: user.authVersion,
    },
    process.env.JWT_SECRET,
    {
      expiresIn: "7d",
    }
  );

const token = req.cookies.token;

if (!token) {
  return res.status(401).json({
    message: "Not authenticated.",
  });
}

const decoded = jwt.verify(token, process.env.JWT_SECRET);

const user = await User.findById(decoded.id).select("-password");

if (!user) {
  return res.status(401).json({
    message: "User no longer exists.",
  });
}

if (user.status === "suspended") {
  return res.status(403).json({
    message: "This account has been suspended.",
  });
}

if ((decoded.authVersion ?? 0) !== user.authVersion) {
  return res.status(401).json({
    message:
      "Your session is no longer valid. Please sign in again.",
  });
}
```

- Audius API Integration

```
async function fetchAudius(endpoint) {
  const response = await fetch(
    `${AUDIUS_API_URL}${endpoint}`,
    {
      headers: {
        "x-api-key": process.env.AUDIUS_API_KEY,
      },
    }
  );

  const data = await response.json();

  if (!response.ok) {
    throw new Error(
      data?.message || "Audius API request failed."
    );
  }

  return data;
}
```

- Playlist API

```
router.post("/", authMiddleware, async (req, res) => {
  try {
    const { name, description } = req.body;

    if (!name || !name.trim()) {
      return res.status(400).json({
        message: "Playlist name is required.",
      });
    }

    const playlist = await Playlist.create({
      user: req.user.id,
      name: name.trim(),
      description: description?.trim() || "",
      songs: [],
    });

    res.status(201).json({
      message: "Playlist created successfully.",
      playlist,
    });
  } catch (error) {
    console.error("Create playlist error:", error);

    res.status(500).json({
      message: "Failed to create playlist.",
    });
  }
});
```

- Likes API

```
router.post('/:songId', authMiddleware, async (req, res) => {
  try {
    const { songId } = req.params;

    const {
      title,
      artist,
      artwork,
    } = req.body;

    // Check if already liked
    const existingLike = await like.findOne({
      user: req.user.id,
      songId,
    });

    if (existingLike) {
      return res.status(400).json({
        message: "Song already liked.",
      });
    }

    const like = await like.create({
      user: req.user.id,
      songId,
      title,
      artist,
      artwork,
    });

    res.status(201).json({
      message: "Song liked successfully.",
      like,
    });

  } catch (error) {
    console.error("Like song error:", error);

    res.status(500).json({
      message: "Failed to like song.",
    });
  }
});
```

- Password Reset OTP

```
const generateOTP = () => {
  return crypto.randomInt(100000, 1000000).toString();
};

/*
 * Hash OTP before storing it.
 */
const hashOTP = (otp) => {
  return crypto
    .createHash("sha256")
    .update(otp)
    .digest("hex");
};
```

Appendix D: Testing Results and user feedback

- Functional Test Cases
- Authentication Testing
- Mood/Search Testing
- Playlist and Likes Testing
- Admin Authorization Testing

ID	Module	Test Case	Expected Result	Status
TC-01	Authentication	Register with valid details	Account successfully created	Pass
TC-02	Authentication	Login with valid credentials	User redirected to dashboard	Pass
TC-03	Authentication	Login with invalid password	Error message displayed	Pass
TC-04	Mood	Select Happy mood	Relevant songs displayed	Pass
TC-05	Search	Search artist name	Matching results displayed	Pass
TC-06	Player	Play a song	Audio begins playing	Pass
TC-07	Player	Pause current song	Playback stops	Pass
TC-08	Playlist	Create playlist	Playlist created successfully	Pass
TC-09	Likes	Like a song	Song appears in liked songs	Pass
TC-10	History	Play a song	Song appears in history	Pass
TC-11	Admin	Login as admin	Admin dashboard accessible	Pass
TC-12	Security	Normal user accesses admin	Access denied	Pass

- Feedback

Timestamp	1. What is your name?	2. How old are you?	3. What is your gender?	4. What is your occupation?	5. Which Verse Hana feature do you like the most?	7. Which feature would you like to see in the next version?	8. Would you recommend Verse Hana to your friends?	9. What did you like the most about Verse Hana?	10. What would you improve about Verse Hana?	11. Did you encounter any issues while using Verse Hana?
25/08/2024 19:34:21	infidafafa	2	2	2	Overall design	Artist profiles	Definitely	artistic	functional	
24/08/2024 10:21:59	Panasikotam Basiet	4	5	5	Mood-based music discovery	Mobile application	Probably	Mood-based music discovery	More songs to the list... Especially ones that are popular must be included	Can't recover account password
24/08/2024 11:58:14	Xin	4	5	4	Other	Artist profiles	Maybe	It was easy to login	It's nice but could be better	No, it was pretty smooth
24/08/2024 12:15:57	Ayyash subelli	5	5	5	Overall design	Download/offline listening	Definitely	its design	download offline	no
24/08/2024 12:48:24	Lakendra Bate	5	5	5	Overall design	Download/offline listening	Definitely	Everything	After clicking it takes time to load the music	Still no
24/08/2024 12:48:16	Yashika Nimsale	4	4	4	Search	Download/offline listening	Probably	Ad free	Theme	No not any
24/08/2024 13:18:18	Roshika poudel	3	4	3	Overall design	Personalized recommendations	Probably	I like simple design	I would add more features for a better user experience	Nope
24/08/2024 13:25:43	Rubina	4	5	5	Lyrics	Download/offline listening	Definitely	Theme	Add Hindi songs also	No not at all
24/08/2024 13:28:01	Jayashika Bhatia	5	5	5	Overall design	Personalized recommendations	Definitely	Everything	Nothing	No
24/08/2024 13:28:32	Jayashika Bhatia	5	5	5	Overall design	Download/offline listening	Definitely	Everything	Nothing	Nope
24/08/2024 13:42:16	Dipika Pokharel	1	2	1	Mood-based music discovery	Download/offline listening	Maybe	Music	All good	No
24/08/2024 13:58:34	Bhishma Thapa	4	4	5	Overall design	Artist profiles	Probably	The design it was cool	Search bar maybe I didn't find it but the music was limited I didn't like it	No 😞
24/08/2024 15:29:29	Siddhartha Gautam	4	2	3	Music player	Other	Probably	Familiar User interface	1. The music search results must show a greater number of songs. 2. Tending page should be collapsible as most users have preference for a clean interface. 3. Pop-up theme card that gives a color choice. 4. The excessive use of emojis is cheapening the feel of the app.	ask if this is bug or intended behavior

User Feedback

Feedback was collected from users who interacted with the Verse Hana platform during development. Users generally found the mood-based discovery approach simple to understand and appreciated the centralized music player, playlist functionality, liked songs, and recently played section.

Some users suggested improvements such as more personalized recommendations, better search capabilities, additional music sources, and mobile application support. These suggestions were considered as potential future enhancements for the platform.

Closing Summary

Verse Hana is a full-stack music streaming platform designed to provide users with a smooth, modern, and interactive music experience. The platform allows users to discover and listen to music, explore different genres and moods, like songs, create and manage playlists, view recently played tracks, and manage their personal profiles. A persistent global music player allows users to continue listening while navigating between different sections of the application.

The platform also includes a dedicated Admin Dashboard that provides centralized management of the application's music ecosystem. Administrators can manage users, artists, songs, moods, and lyrics, as well as synchronize music and artist data through the Audius catalog integration.

From a technical perspective, Verse Hana demonstrates practical full-stack development through a React-based frontend, backend API integration, database-driven content management, authentication and protected routes, state management, music playback, and responsive interface design. The project also incorporates user activity such as likes and listening history, providing a foundation for future personalization and recommendation features.

Overall, Verse Hana has grown from a simple music-player concept into a complete music-platform project with user functionality, content management, discovery, administration, and persistent playback. Future improvements can include local music uploads, advanced search, personalized recommendations, a more dynamic user-driven trending system, improved analytics, and further performance and security optimization.